UNIT KOMPETENSI 03 • TI.PG.03.2026

MENERAPKAN PERINTAH EKSEKUSI BAHASA PEMROGRAMAN BERBASIS TEKS, GRAFIK, DAN MULTIMEDIA

Implementasi Riil pada Proyek: Portal Karir, Profil Industri & Modul Automation-GPT (Fullstack Web)

Presenter / Mahasiswa	Aditya Ristanto Mukti (Fullstack Web Developer Intern)	**Repository**	`github.com/AdityaRistantoMukti/Website_Intersis`
Institusi / Mitra	PT Global Philsonic Technologies (Intersis) Batam	**Tech Stack Utama**	React 19, Vite, Tailwind CSS v4, Node.js Express 5, MySQL2
Cakupan Outline Sub-Title	Mekanisme running source code • Debugging • Pengelolaan executable files		

I. PETA OUTLINE & STRUKTUR PEMBAHASAN

Unit kompetensi **Menerapkan Perintah Eksekusi Bahasa Pemrograman Berbasis Teks, Grafik, dan Multimedia** ini disusun secara berkesinambungan mencakup seluruh sub-judul yang didefinisikan pada diagram alur kompetensi, diaplikasikan langsung pada arsitektur website yang telah dibangun:

- **Mekanisme running source code**: Mengonfigurasi dan mengoperasikan eksekusi runtime Node.js v18+, bundler Vite ESM, skrip npm, serta orkestrasi cloud hosting.
- **Debugging**: Mendiagnosis kesalahan sintaksis, logika, network status, serta penggunaan middleware logger terpusat dan DevTools.
- **Pengelolaan executable files**: Memproses stream upload media (gambar/dokumen), file system static serving, serta automasi CLI rendering headless Chrome via Puppeteer.

II. NASKAH WICARA LENGKAP PRESENTER (VERBATIM SPEECH SCRIPT)

🎤 PANDUAN NARASI WICARA PRESENTER (SPEAKING SCRIPT)

[Pembukaan & Sikap] "Selamat pagi/siang Dewan Penguji. Pada unit kompetensi **Menerapkan Perintah Eksekusi Bahasa Pemrograman Berbasis Teks, Grafik, dan Multimedia**, saya akan mendemonstrasikan bagaimana siklus hidup kode, dari proses eksekusi lokal hingga pemrosesan aset multimedia, dikelola secara profesional."

[Menjelaskan Sub 1: Mekanisme Running Source Code] "Dalam *Mekanisme Running Source Code*, proyek kami menerapkan arsitektur client-server dual-runtime. Di sisi backend, kami menjalankan runtime **Node.js v24** dengan framework **Express v5** melalui perintah `npm run dev` yang memanfaatkan `nodemon` untuk hot-reloading otomatis saat ada perubahan berkas. Di sisi frontend, kami mengeksekusi **Vite** berbasis Rollup ESM engine yang memproses Hot Module Replacement (HMR) dalam hitungan milidetik. Untuk fase produksi, perintah `vite build` mengompilasi kode JSX dan CSS menjadi static chunks yang sangat teroptimasi untuk di-deploy ke Vercel dan Railway."

[Menjelaskan Sub 2: Debugging] "Pada aspek *Debugging*, kami menerapkan pendekatan observabilitas berlapis. Di backend, kami mengintegrasikan middleware `morgan` untuk melacak setiap HTTP Request secara transparan dan membangun Global Error Handling Middleware yang menangkap exception database tanpa membiarkan server crash. Di frontend, kami memanfaatkan Chrome DevTools Network Tab untuk memvalidasi payload JSON dan header status HTTP (200 OK, 401 Unauthorized, 422 Unprocessable Entity), serta React DevTools untuk menginspeksi state component secara real-time."

[Menjelaskan Sub 3: Pengelolaan Executable Files & Multimedia] "Pada sub-topik *Pengelolaan Executable Files*, kami menangani pemrosesan aset grafis dan multimedia secara ketat. Kami menggunakan `multer` untuk memfilter tipe file MIME (hanya mengizinkan JPEG, PNG, dan WebP) dengan batasan ukuran maksimum 5MB. Selain itu, kami juga merancang skrip automasi executable `generate_handbook_pdf.js` yang memanfaatkan Puppeteer headless browser untuk mengonversi dokumen HTML kompleks menjadi berkas PDF A4 berstandar resmi secara otomatis melalui baris perintah CLI."

III. PEMBAHASAN TEKNIS RINCI & BEDAH KODE NYATA PROYEK

Berikut adalah pembuktian implementasi teknis dari masing-masing sub-judul berdasarkan baris kode dan berkas nyata di dalam repository proyek:

3.1. Mekanisme Eksekusi Runtime & Orkestrasi Skrip NPM

Kaitkan ke Codebase: **`backend/package.json & frontend/package.json`**

Lingkungan eksekusi dibagi menjadi dua modul runtime independen yang dapat dijalankan secara bersamaan:

Layer	Runtime & Bundler	Perintah Eksekusi	Perilaku Runtime
Backend Service	Node.js v18+ (CommonJS) / Express 5	`npm run dev` (Nodemon)	Memantau direktori `backend/src`, merestart server otomatis saat terjadi perubahan kode pada port 5000.
Frontend Client	Vite 8.3 / React 19 (ES Modules)	`npm run dev` (Vite HMR)	Menyediakan dev-server instan berbasis native browser ESM pada port 5173 dengan pembaruan state tanpa refresh halaman.
Production Build	Rollup Bundler Engine	`npm run build`	Minifikasi JavaScript, tree-shaking CSS Tailwind, dan menghasilkan berkas static di direktori `frontend/dist`.

backend/package.json

```
"scripts": {
  "dev": "nodemon src/server.js",
  "start": "node src/server.js",
  "seed": "node src/seeders/seed.js",
  "test": "node test_api_runner.js"
}
```

3.2. Strategi Debugging, HTTP Logging, dan Exception Handling

Kaitkan ke Codebase: **`backend/src/server.js & backend/src/middlewares/errorHandler.js`**

Debugging dilakukan secara terpusat untuk memastikan tidak ada kesalahan internal yang membocorkan kredensial sistem ke publik:

- **HTTP Traffic Monitoring:** Menggunakan `morgan('dev')` untuk mencatat method, URL rute, kode status response, dan response time milidetik pada konsol terminal backend.
- **Centralized Error Interceptor:** Middleware Express yang menangani error penolakan promise database, duplikasi key, dan kegagalan token autentikasi.

backend/src/server.js

```
// Global Error Handling Middleware untuk Debugging Terpusat
app.use((err, req, res, next) => {
  console.error('[SERVER EXCEPTION]', {
    timestamp: new Date().toISOString(),
    path: req.originalUrl,
    method: req.method,
    message: err.message,
    stack: process.env.NODE_ENV === 'development' ? err.stack : undefined
  });

  return res.status(err.status || 500).json({
    success: false,
    message: err.message || 'Terjadi kesalahan internal pada server.',
    error_code: err.code || 'INTERNAL_ERROR'
  });
});
```

3.3. Pengelolaan Upload Media & Automasi Executable PDF Generator

Kaitkan ke Codebase: **`backend/src/middlewares/upload.js & generate_handbook_pdf.js`**

Pengelolaan file grafis multimedia dan skrip executable mencakup dua komponen utama:

1. **Middleware Multer Storage:** Mengelola stream file upload foto profil dan galeri industri. File diverifikasi ekstensinya, dinamai ulang menggunakan kombinasi timestamp acak, dan disimpan ke direktori publik `/uploads`.
2. **Headless Chrome Automation (Puppeteer):** Skrip Node.js mandiri yang mengontrol browser Chrome secara programmatic untuk mem-print dokumen HTML teknis menjadi berkas PDF A4 beresolusi tinggi tanpa intervensi manual.

generate_handbook_pdf.js

```
// Automasi Pembuatan Berkas PDF Menggunakan Puppeteer-Core
const puppeteer = require('./frontend/node_modules/puppeteer-core');
const browser = await puppeteer.launch({
  executablePath: 'C:\\Program Files\\Google\\Chrome\\Application\\chrome.exe',
  headless: 'new',
  args: ['--no-sandbox', '--disable-setuid-sandbox', '--disable-gpu']
});
const page = await browser.newPage();
await page.goto('file:///' + htmlPath.replace(/\\/g, '/'), { waitUntil: 'networkidle0' });
await page.pdf({
  path: outputPath,
  format: 'A4',
  margin: { top: '16mm', bottom: '16mm', left: '16mm', right: '16mm' },
  printBackground: true
});
await browser.close();
```

IV. POIN KUNCI PRESENTASI (SLIDE TALKING POINTS)

POINTER RINGKAS UNTUK TAMPILAN SLIDE / LAYAR

- Dual Runtime Ecosystem: Node.js v24 (Backend Express) dan Vite HMR Engine (Frontend React).
- Skrip Eksekusi Terstruktur: 'npm run dev', 'npm run build', dan 'npm run seed'.
- Debugging Observability: Logging HTTP otomatis dengan Morgan dan Centralized Error Handler.
- Multimedia Upload Pipeline: Validasi MIME-type dan proteksi ukuran file via Multer.
- Headless Browser Automation: Generator PDF otomatis menggunakan Puppeteer dan Chrome Binary Engine.

V. MATRIKS TANYA JAWAB & PANDUAN PEMBELAAN SIDANG (Q&A DEFENSE)

Prediksi pertanyaan evaluasi dari Penguji / Asesor / Pembimbing Industri beserta jawaban teknis yang siap dipertahankan:

Q1: Mengapa Anda menggunakan 'puppeteer-core' dan mengarahkan executablePath ke Chrome lokal, bukan 'puppeteer' standar?

Jawaban Pembelaan Presenter: Penggunaan puppeteer-core menghindarkan pengunduhan binary Chromium tambahan sebesar 300MB ke dalam node_modules. Kami memanfaatkan executable Chrome resmi yang sudah terpasang di sistem operasi host, sehingga eksekusi lebih cepat, hemat bandwidth, dan hemat kapasitas penyimpanan.

Q2: Bagaimana Anda mencegah celah keamanan saat pengguna mengunggah file media ke server?

Jawaban Pembelaan Presenter: Kami menerapkan fileFilter pada Multer untuk memeriksa header MIME type (hanya mengizinkan image/jpeg, image/png, image/webp), membatasi ukuran maksimal 5MB, serta mengubah nama asli file menjadi hash timestamp acak untuk mencegah penimpaan file sistem atau eksekusi remote script (.php/.sh).